হালখাতা ডিজিটাল (Halkhata Digital)

A Digital Ledger Management System for Small and Medium Businesses

Internet & Web Programming Project Report

Course Code: CSE - 3109

Course Title: Internet & Web Programming

Submission Date: June 5, 2026

Submitted To

Md. Aminul Islam

Associate Professor

Department of Computer Science and Engineering,

Jagannath University, Dhaka

Submitted By

Name	*Student ID*
Tasfia Tasnim	B220305017
Atik Jawad	B220305043

Department of Computer Science and Engineering

Jagannath University, Dhaka, Bangladesh

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project supervisor, **Md. Aminul Islam,** for the continuous guidance, valuable feedback, and encouragement provided throughout the development of this project. His insights on web development practices and system design significantly shaped the direction of this work.

We are also thankful to the faculty members of the Department of Computer Science and Engineering at Jagannath University, for creating an academic environment that encouraged independent problem solving and practical application of classroom concepts to a real-world business scenario.

This project would not have been possible without the everyday observations of how small shop owners in Bangladesh manage customer credit using the traditional paper Tally Book. Conversations with local shopkeepers about the difficulties of tracking dues, calculating monthly totals, and locating old entries directly inspired the core objectives of this system.

Finally, We would like to thank our family and friends for their patience and support during the course of this project, and our peers for their helpful discussions during testing and review of the application.

Table of Contents

Chapter 1: Introduction 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Motivation 3

1.5 Scope 3

1.6 Expected Outcome 4

Chapter 2: System Analysis 6

2.1 Existing System 6

2.2 Limitations of the Existing System 6

2.3 Proposed System 7

2.4 Requirement Analysis 7

2.4.1 Functional Requirements 7

2.4.2 Non-Functional Requirements 9

2.5 Feasibility Study 9

Chapter 3: System Design 11

3.1 Overall Architecture 11

3.2 Frontend Architecture 12

3.3 Backend Architecture 12

3.4 Client-Server Communication 12

3.5 Database Design 13

3.5.1 Entity Relationship Diagram 13

3.5.2 Database Tables 14

3.5.3 Relationships 15

3.6 Folder Structure 16
3.7 API Structure 16
3.8 Technology Justification 17
3.8.1 Why SQLite 17
3.8.2 Why Express.js 17
3.8.3 Why Vanilla JavaScript 17
3.9 Security Design 18
Chapter 4: Technology Stack 19
4.1 Frontend Technologies 19
4.1.1 HTML5 19
4.1.2 CSS3 19
4.1.3 JavaScript (Vanilla, ES6+) 20
4.1.4 Chart.js 20
4.2 Backend Technologies 20
4.2.1 Node.js 20
4.2.2 Express.js 21
4.3 Database 21
4.3.1 SQLite (via better-sqlite3) 21
4.4 Supporting Libraries 22
4.4.1 bcryptjs 22
4.4.2 express-session 22
4.4.3 Multer 22
4.4.4 UUID 23
4.5 Development and Version Control Tools 23
4.5.1 Git and GitHub 23
4.6 Technology Stack Summary 24
Chapter 5: Implementation 25

5.1 Customer Management 25
5.2 Supplier Management 26
5.3 Inventory Management 26
5.4 Sales Recording 27
5.5 Credit Management 27
5.6 Repayment System 28
5.7 Expense Management 29
5.8 Dashboard 29
5.9 Reports (Monthly and Yearly) 30
5.10 CSV Export 30
5.11 PDF Export 31
5.12 Search 32
5.13 Filtering 32
5.14 Settings 33
5.15 Authentication 33
5.16 Image Upload 34
5.17 Database Operations 35
5.18 Error Handling 35
5.19 Validation 36
Chapter 6: Testing 37
6.1 Manual Testing Approach 37
6.2 Test Environment 37
6.3 Test Cases 37
6.4 Validation Testing 39
6.5 Summary of Testing Results 40
Chapter 7: Results 41
7.1 Overview of the Completed Application 41

7.2 Application Screenshots....42
Chapter 8: Conclusion....49
8.1 Summary....49
8.2 Achievements....49
8.3 Limitations....50
8.4 Lessons Learned....50
Chapter 9: Future Work....51

LIST OF FIGURES

Figure 3.1: System Architecture Diagram 11

Figure 3.2: Entity Relationship Diagram 13

Figure 7.1: Login Screen 42

Figure 7.2: Registration Screen 42

Figure 7.3: Dashboard Overview 43

Figure 7.4: Customer Management Page 43

Figure 7.5: Inventory Management Page 44

Figure 7.6: Supplier Ledger Page 45

Figure 7.7: Expense Overview Page 46

Figure 7.8: Monthly Report View 46

Figure 7.9: Yearly Report View 47

Figure 7.10: Settings Page 48

Figure 7.12: CSV Export Result 48

LIST OF TABLES

Table 2.1: Functional Requirements 7

Table 2.2: Non-Functional Requirements 9

Table 3.1: Database Tables Overview 14

Table 3.2: Key Entity Relationships 15

Table 4.1: Technology Stack Summary 24

Table 6.1: Manual Test Cases 37

Table 6.2: Validation Test Summary 39

Chapter 1: Introduction

1.1 Background

In Bangladesh, small grocery shops, pharmacies, hardware stores, and roadside retail businesses have long depended on selling goods to regular customers on credit. This informal credit relationship, recorded in a handwritten notebook known as the হালখাতা (halkhata), is a cultural and economic institution that builds trust between a shopkeeper and the surrounding community. Each page of the halkhata typically records a customer's name, the items purchased on credit, the date, and running notes of partial repayments made over time.

While this system has served small businesses for generations, it is inherently fragile. A single misplaced or water-damaged notebook can erase months of financial history. Handwriting can be misread, arithmetic errors can accumulate unnoticed, and locating a specific customer's balance among dozens of pages is time-consuming. As mobile phones and low-cost internet access have become widely available even in semi-urban and rural areas of Bangladesh, there is a growing opportunity to replace this paper-based process with a digital equivalent that keeps the same simplicity but removes the risk of data loss and manual calculation error.

হালখাতা ডিজিটাল (Halkhata Digital) was conceived as a direct response to this opportunity. Rather than building a generic point-of-sale or accounting suite, the project focuses specifically on the day-to-day workflow of a small shop owner: recording a sale, marking whether it was paid in cash or added to a customer's running balance, and later recording a repayment against that balance. Every other feature in the system — inventory tracking, supplier ledgers, expense recording, and financial reporting — was built around this central workflow.

1.2 Problem Statement

Manual ledger-keeping creates a set of recurring operational problems for small shop owners in Bangladesh:

- Physical ledgers can be lost, torn, or damaged by moisture, and there is no backup copy of the data they contain.
- Calculating a customer's total outstanding balance requires manually reviewing every entry made for that customer across potentially many pages.

- There is no simple way to generate a monthly or yearly summary of total sales, total collections, or total expenses without redoing all the arithmetic by hand.
- Shop owners have no objective way of identifying which customers are reliably repaying credit and which are becoming a repeated risk.
- Stock levels are tracked separately from sales, if at all, making it difficult to know which products are running low or nearing expiry.
- Supplier dues — money the shop owner owes to their own wholesalers — are often tracked informally or not tracked at all, leading to disputes.
- None of the above information can be exported, shared, or backed up externally.

These problems are not unique to any single shop; they are structural limitations of paper as a medium for financial record-keeping. A digital system that mirrors the mental model of a halkhata — a customer, a running balance, and a history of transactions — while adding automatic calculation, search, backup, and reporting capabilities, directly addresses this problem without asking the shop owner to abandon a workflow they already understand.

1.3 Objectives

The project was guided by the following core objectives:

1. Design and implement a web-based ledger system that allows a shop owner to record customer credit sales, cash sales, and repayments with minimal data entry effort.

2. Automatically calculate and display each customer's current outstanding balance, eliminating manual arithmetic.

3. Provide a batch-aware inventory system that tracks stock quantity, cost price, selling price, and expiry date at the level of individual purchase batches.

4. Maintain a supplier ledger that mirrors the customer ledger, allowing the shop owner to track amounts owed to their own suppliers.

5. Generate structured monthly and yearly financial reports, including revenue, expenses, net profit, and visual charts of key trends.

6. Allow exporting of financial data as CSV spreadsheets and print-ready PDF documents for offline record-keeping or sharing with an accountant.

7. Secure each shop owner's data behind an individual account protected by a PIN, ensuring that data belonging to one business is never visible to another.

8. Support multiple branches or shop locations under a single account, with the ability to filter all data by branch.

1.4 Motivation

The motivation for this project came from direct observation of local shop owners struggling with the limitations described above. Existing international ledger applications are frequently designed around markets and payment ecosystems that do not map cleanly onto Bangladeshi retail habits — for example, the local convention of tracking mobile financial services such as bKash, Nagad, and Rocket as separate cash accounts is rarely represented well in generic applications. Building the system from the ground up made it possible to design the data model, terminology, and workflow around what a Bangladeshi shopkeeper actually does every day, rather than adapting a foreign template.

There was also a strong technical motivation: to demonstrate that a complete, production-quality business application can be built using a comparatively small and well-understood set of technologies — a relational database, a Node.js backend, and a dependency-light vanilla JavaScript frontend — without requiring a heavyweight frontend framework or third-party backend-as-a-service platform. This approach keeps the codebase approachable, easy to audit, and inexpensive to host.

1.5 Scope

The scope of হালখাতা ডিজিটাল (Halkhata Digital) covers the full lifecycle of a small shop's day-to-day bookkeeping. The implemented system includes:

- Customer registration and management, including contact details, credit limits, and a computed trust score.
- Recording of cash sales and credit sales, with automatic linkage to inventory when a product is sold.
- Recording of customer repayments, automatically applied against the oldest outstanding dues first.

- Batch-based inventory management, allowing multiple purchase batches of the same product to be tracked individually while being displayed to the user as a single aggregated stock entry.
- Supplier management and a supplier transaction ledger for tracking amounts owed to wholesalers.
- Expense recording, categorized by type (for example, rent, transportation, salaries, and utility bills).
- A dashboard summarizing key business indicators at a glance.
- Monthly and yearly financial reports with interactive charts, product analytics, customer analytics, and business insights.
- CSV and PDF export of reports for offline use.
- Multi-branch support, allowing a single account to manage more than one physical shop location.
- Authentication using a phone number or email address combined with a four-digit PIN.
- A settings area for managing shop details, branches, PIN changes, and data backup or restore.

The following areas were consciously excluded from the current scope and are discussed separately in Chapter 9 (Future Work): native mobile applications, offline-first synchronization, barcode and QR-code based inventory scanning, SMS-based customer reminders, multi-employee accounts with role-based permissions, and AI-based analytics or predictions.

1.6 Expected Outcome

By the completion of this project, the system was expected to function as a reliable replacement for a shop owner's paper ledger for all core bookkeeping tasks. Specifically, the working application should allow a shop owner to add a new customer and record a transaction within seconds, view an always-current balance for every customer without manual calculation, generate an accurate monthly or yearly financial summary on demand, and retain a complete, backed-up history of every transaction ever recorded. The resulting prototype is intended to serve both as a genuinely usable tool for a small business and as a demonstration of applying

structured software engineering practice — requirement analysis, system design, iterative implementation, and testing — to a real, locally relevant problem.

Chapter 2: System Analysis

2.1 Existing System

The existing system used by the overwhelming majority of small shop owners in Bangladesh is the handwritten halkhata notebook, supplemented in some cases by informal notes on scraps of paper or verbal agreements with regular customers. A small number of shop owners have adopted generic mobile ledger applications available on app stores; however, these applications are typically designed as general-purpose "credit book" apps and do not integrate inventory, supplier tracking, multi-branch operation, or locally relevant payment methods into a single coherent system. Where such applications are used, shop owners often continue to maintain a parallel paper record because they do not fully trust the digital tool, or because the digital tool does not cover their supplier-side bookkeeping at all.

2.2 Limitations of the Existing System

Analysis of the existing manual and semi-digital practices revealed the following recurring limitations:

- No backup: a lost or damaged notebook results in permanent loss of financial history.
- No automatic calculation: every balance, monthly total, and yearly total must be computed by hand.
- No search capability: finding a specific customer's history requires manually flipping through pages.
- No reporting: there is no way to generate a structured summary of business performance over a period of time.
- No inventory linkage: sales are not connected to stock levels, so shop owners frequently discover stock shortages only when a customer asks for an item that is unavailable.
- No supplier-side tracking: amounts owed to wholesalers are tracked informally, if at all, leading to disagreements over outstanding dues.
- No data portability: information cannot be exported, printed in a structured format, or shared electronically with an accountant or family member.

- Generic digital alternatives, where used, do not reflect local payment habits (cash, bKash, Nagad, Rocket, bank transfer) or the specific structure of a Bangladeshi retail credit relationship.

2.3 Proposed System

The proposed system, হালখাতা ডিজিটাল (Halkhata Digital), addresses each of the limitations identified above by digitizing the ledger while preserving its conceptual simplicity. Every customer has a single profile with a continuously updated balance; every transaction, whether a cash sale, a credit sale, or a repayment, is recorded as an individual, timestamped entry linked to that customer. The system automatically maintains running totals, so a shop owner never needs to perform manual addition or subtraction to know how much a customer currently owes.

Beyond replicating the paper ledger, the proposed system extends the concept in ways that are only practical in a digital medium: it computes a trust score for each customer based on their repayment history, tracks inventory at the level of individual purchase batches so that cost and expiry information is never lost when new stock is added, maintains a parallel ledger for the shop's own suppliers, and generates monthly and yearly financial reports with charts that would be extremely time-consuming to reproduce by hand. All data is stored in a structured SQLite database rather than loose paper pages, and the entire dataset can be exported as a JSON backup file at any time.

2.4 Requirement Analysis

2.4.1 Functional Requirements

Table 2.1 summarizes the primary functional requirements identified for the system.

ID	Requirement	Description
FR-01	User Registration and Login	The system shall allow a shop owner to register an account using their name, shop name, and a phone number or email address paired with a PIN, and to log in using the same credentials.
FR-02	Customer Management	The system shall allow creating, viewing, editing, and deleting customer records, including name, phone number, address, and credit limit.

ID	Requirement	Description
FR-03	Sales Recording	The system shall allow recording a cash sale or a credit sale, optionally linked to a product from inventory.
FR-04	Repayment Recording	The system shall allow recording a customer repayment and automatically apply it against the customer's oldest outstanding dues.
FR-05	Inventory Management	The system shall allow adding products and purchase batches, tracking quantity, buy price, sell price, and expiry date per batch.
FR-06	Supplier Management	The system shall allow recording suppliers and a ledger of amounts owed to and paid to each supplier.
FR-07	Expense Management	The system shall allow recording business expenses with a category, amount, and payment method.
FR-08	Dashboard	The system shall display a summary dashboard showing key totals, recent transactions, and top outstanding customers.
FR-09	Monthly and Yearly Reports	The system shall generate monthly and yearly financial reports with revenue, expense, profit, and chart-based visualizations.
FR-10	Data Export	The system shall allow exporting reports as CSV files and as print-ready PDF documents.
FR-11	Branch Management	The system shall allow a single account to manage multiple shop branches and filter data by branch.
FR-12	Data Backup and Restore	The system shall allow exporting the full dataset as a JSON file and restoring from a previously exported file.

Table 2.1: Functional Requirements

2.4.2 Non-Functional Requirements

ID	Requirement	Description
NFR-01	Usability	The interface shall be simple enough for a shop owner with limited technical experience to use without formal training.
NFR-02	Performance	Common operations such as recording a transaction or loading a customer list shall complete within a short, perceptible delay under normal load.
NFR-03	Data Integrity	Financial calculations, including balances and report totals, shall remain accurate and consistent across all screens.
NFR-04	Security	User credentials shall be stored using one-way password hashing, and each account's data shall be isolated from other accounts.
NFR-05	Maintainability	The codebase shall be organized into clear modules (routes, database access, and frontend logic) to simplify future changes.
NFR-06	Portability	The application shall run on any modern web browser without requiring the installation of a native application.
NFR-07	Localization	The user interface shall support the Bengali language and Bengali numeral formatting throughout.

Table 2.2: Non-Functional Requirements

2.5 Feasibility Study

Technical Feasibility: The technologies selected for this project — HTML5, CSS3, vanilla JavaScript, Node.js, Express.js, and SQLite — are mature, well-documented, and freely available. No specialized hardware or paid infrastructure is required for development, and the resulting application can be hosted on modest, low-cost server infrastructure. The project team's existing familiarity with JavaScript across both the frontend and backend removed the need to learn an entirely new language, making the technical approach realistic within the available project timeline.

Operational Feasibility: Because the system was designed to mirror the mental model of a paper halkhata rather than introduce unfamiliar accounting terminology, the learning curve for a shop owner is expected to be low. Core actions — adding a customer, recording a sale,

recording a repayment — map directly onto tasks the user already performs manually, which supports realistic day-to-day adoption.

Economic Feasibility: The entire technology stack used in this project is open-source and free to use, and the application has no dependency on paid third-party services for its core functionality. This keeps both development and future hosting costs low, which is an important consideration given that the target users are small businesses operating on thin margins.

Schedule Feasibility: The project was developed iteratively, beginning with a minimal working prototype covering customer and transaction management, followed by incremental addition of inventory, supplier, expense, and reporting modules. This incremental approach allowed a functional system to exist at every stage of development, reducing schedule risk compared to a single large release at the end of the project.

Chapter 3: System Design

3.1 Overall Architecture

হালখাতা ডিজিটাল (Halkhata Digital) follows a classic three-tier client-server architecture consisting of a browser-based presentation layer, a Node.js and Express.js application layer, and an SQLite persistence layer. The browser communicates with the backend exclusively through a REST-style HTTP API that returns JSON data; there is no server-side page rendering beyond serving the initial static HTML shell. This separation keeps the frontend and backend loosely coupled: the backend has no knowledge of how data will be displayed, and the frontend has no knowledge of how data is stored.

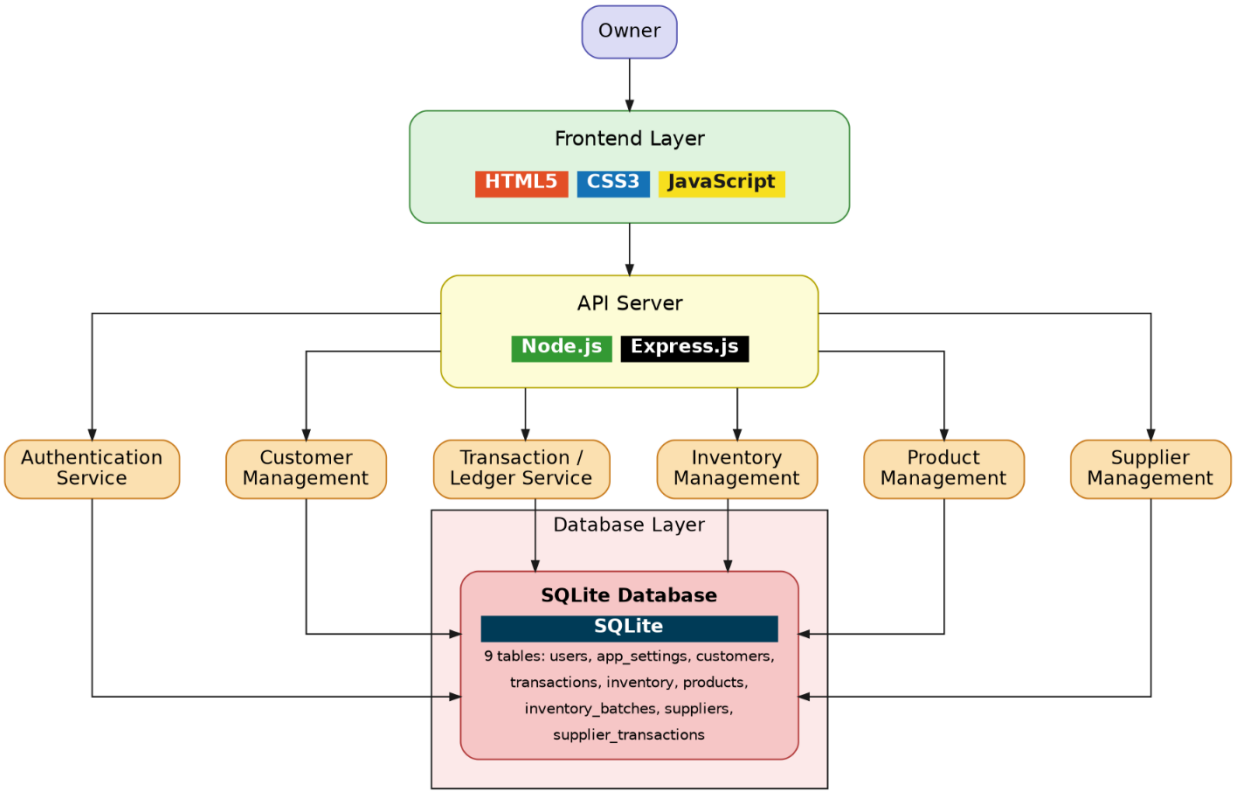


Figure 3.1: System Architecture Diagram

At a high level, a request originates from a user action in the browser (for example, submitting the "Add Transaction" form), is sent as an authenticated HTTP request to the Express.js server, validated and processed by the relevant route handler, persisted or retrieved via the SQLite database, and the resulting JSON response is used by the frontend JavaScript to update the visible interface without a full page reload.

3.2 Frontend Architecture

The frontend is implemented as a single-page application written in vanilla JavaScript, without a framework such as React or Vue. The interface is organized around a single HTML shell (index.html) containing a sidebar navigation menu and a set of page containers, one for each major section of the application — Dashboard, Customers, Transactions, Inventory, Suppliers, Expenses, Overdue, Reports, and Settings. Navigation between pages is handled entirely on the client side by toggling the visibility of these containers and calling the appropriate data-loading function, rather than requesting a new HTML document from the server.

Reusable interface behavior — such as modals, toast notifications, searchable dropdown components, and chart rendering helpers — is implemented as shared JavaScript functions rather than framework components. This keeps the dependency footprint small and the runtime behavior easy to trace, at the cost of requiring more explicit state management than a component-based framework would provide.

3.3 Backend Architecture

The backend is implemented using Express.js and organized around a set of route handlers grouped by resource: authentication, customers, transactions, products and inventory batches, suppliers, supplier transactions, expenses, accounts, and reports. Each route handler is responsible for validating incoming request data, applying business rules (for example, verifying sufficient stock before completing a sale, or preventing deletion of the last remaining branch), and issuing the corresponding SQL statements through a shared database connection module.

Session-based authentication middleware runs before every protected route, attaching the authenticated user's identifier to the request object. Every subsequent database query in that request is scoped to this identifier, which is the mechanism by which the system guarantees that one shop owner's data is never visible to another.

3.4 Client-Server Communication

All communication between the frontend and backend uses standard HTTP methods over a JSON-based REST interface: GET for retrieving data, POST for creating new records, PUT for updating existing records, and DELETE for removing records. A lightweight wrapper

function on the frontend (referred to internally as the API helper) centralizes request construction, attaches the necessary session cookie, and standardizes error handling, so that individual page scripts can call a resource endpoint and receive either parsed JSON data or a thrown error without repeating boilerplate code for every request.

3.5 Database Design

3.5.1 Entity Relationship Diagram

The relational structure of the system is illustrated conceptually in Figure 3.2.

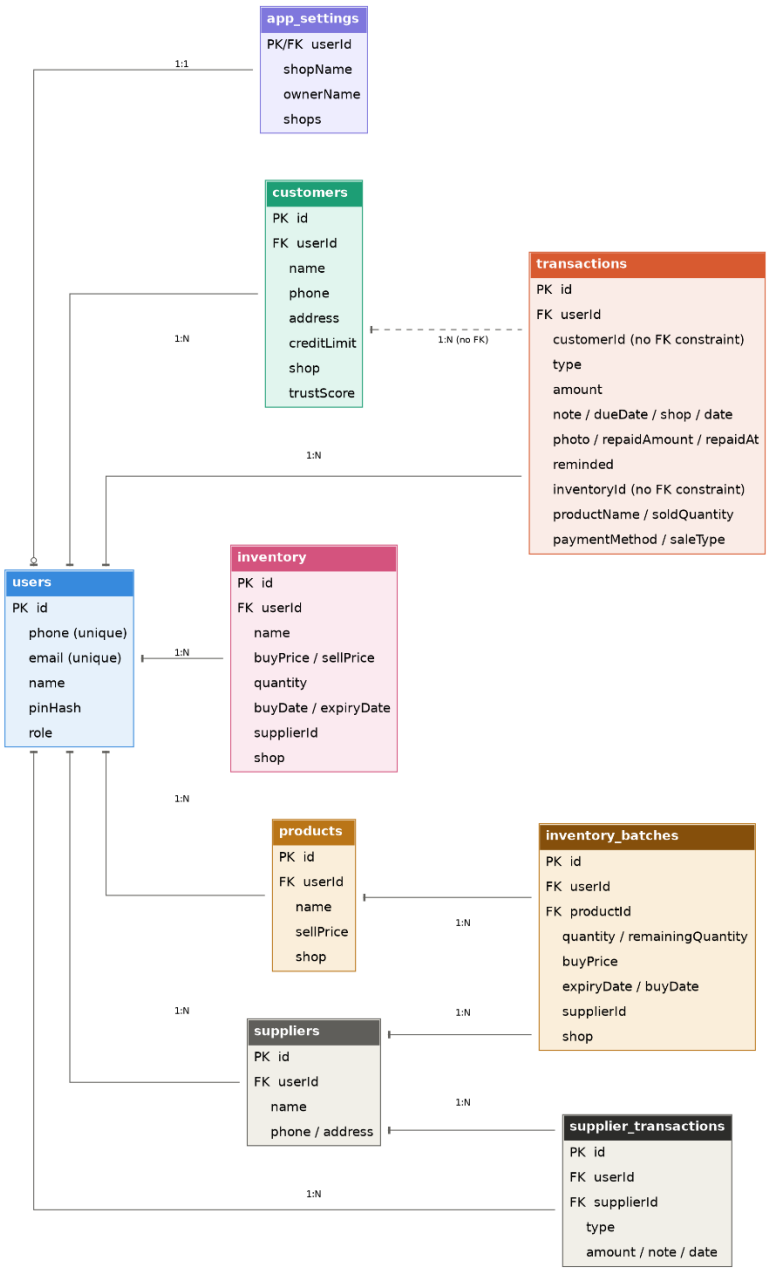


Figure 3.2: Entity Relationship Diagram

3.5.2 Database Tables

Table 3.1 summarizes the primary tables used in the SQLite database and their purpose.

Table	Key Fields	Purpose
users	id, phone, email, name, pinHash	Stores registered shop owner accounts and hashed PIN credentials.
app_settings	userId, shopName, ownerName, shops	Stores per-account shop configuration, including the list of branches.
customers	id, userId, name, phone, creditLimit, trustScore	Stores each shop owner's registered customers.
transactions	id, userId, customerId, type, amount, dueDate, repaidAmount	Stores every cash sale, credit sale, and repayment event.
products	id, userId, name, sellPrice, shop	Stores the aggregate product catalog for inventory.
inventory_batches	id, productId, quantity, remainingQuantity, buyPrice, expiryDate	Stores individual purchase batches belonging to a product.
suppliers	id, userId, name, phone	Stores registered suppliers for the shop.
supplier_transactions	id, supplierId, type, amount, date	Stores amounts owed to and paid to each supplier.
expenses	id, userId, title, category, amount, date	Stores recorded business expenses.
accounts	id, userId, name, type, balance	Stores cash and mobile-financial-service balances (cash, bKash, Nagad, Rocket, bank).
account_transactions	id, accountId, type, amount, relatedId	Stores a ledger of movements into and out of each account.

Table 3.1: Database Tables Overview

3.5.3 Relationships

Table 3.2 describes the key relationships enforced between tables through foreign keys.

Relationship	Type	Description
users → customers, products, suppliers, expenses, accounts	One-to-Many	Every business record belongs to exactly one user account, enforcing complete data isolation between shop owners.
customers → transactions	One-to-Many	Each customer can have any number of cash sale, credit sale, and repayment transactions.
products → inventory_batches	One-to-Many	A product's total stock is the sum of the remaining quantity across all of its batches.
suppliers → supplier_transactions	One-to-Many	Each supplier has its own running ledger of amounts owed and paid.
accounts → account_transactions	One-to-Many	Every change to a cash or mobile-service balance is logged as an individual account transaction.
transactions → inventory_batches	Indirect (via FIFO deduction)	A credit or cash sale that references a product deducts stock from one or more batches, earliest expiry first.

Table 3.2: Key Entity Relationships

All foreign key relationships involving a user's own data are configured with cascading deletes, so that removing a customer or product also removes its dependent transaction or batch records, preventing orphaned data from accumulating in the database.

3.6 Folder Structure

The project repository is organized into a small number of top-level directories, keeping backend and frontend code clearly separated:

```
halkhata/
├── server.js          (Express application entry point and route definitions)
├── db.js              (SQLite schema, migrations, and shared query helpers)
├── package.json       (Project metadata and dependencies)
├── data/
│   └── database.db    (SQLite database file)
├── uploads/           (Uploaded receipt and expense photos)
└── public/
    ├── index.html     (Main single-page application shell)
    ├── style.css      (Global stylesheet)
    ├── script.js      (All frontend logic and API calls)
    ├── login.html     (Login screen)
    └── signup.html    (Registration screen)
```

3.7 API Structure

The backend exposes a resource-oriented REST API grouped by functional area. The main endpoint groups are summarized below:

- Authentication: /api/auth/signup, /api/auth/login, /api/auth/check, /api/auth/info, /api/auth/setup, /api/auth/change-pin
- Customers: /api/customers, /api/customers/:id, /api/customers/:id/transactions, /api/customers/:id/analytics
- Transactions and Sales: /api/transactions, /api/sales
- Products and Inventory: /api/products, /api/products/:id/batches, /api/batches/:id
- Suppliers: /api/suppliers, /api/suppliers/:id, /api/supplier-transactions
- Expenses: /api/expenses, /api/expenses/:id, /api/expenses/:id/photo
- Accounts: /api/accounts, /api/accounts/:id, /api/accounts/:id/transactions
- Dashboard and Reports: /api/dashboard, /api/report/monthly, /api/report/monthly/:year/:month, /api/overdue
- Data Management: /api/backup, /api/restore, /api/seed

Every route in the list above, with the exception of the authentication endpoints themselves, is protected by session middleware and automatically scoped to the currently authenticated user.

3.8 Technology Justification

3.8.1 Why SQLite

The project originally stored data in flat JSON files, one per resource type. This approach became difficult to maintain as the number of related entities grew, since every read or write operation required loading, parsing, and re-serializing an entire file, and there was no mechanism to enforce relationships between records. SQLite was selected as a replacement because it provides a genuine relational schema with foreign keys, indexes, and transactional writes, while still requiring no separate database server process — the entire database lives in a single file on disk, which matches the lightweight deployment goals of the project. The migration from JSON files to SQLite also made it straightforward to add features such as automatic FIFO stock deduction and cross-table trust-score calculation, which would have been considerably more error-prone using hand-written JSON manipulation.

3.8.2 Why Express.js

Express.js was chosen for the backend because it provides a minimal, unopinionated routing and middleware layer on top of Node.js without imposing a rigid application structure. This allowed the route handlers to be organized around the natural resources of the domain (customers, transactions, inventory, and so on) rather than around a framework-specific folder convention, and it kept the total number of external dependencies small.

3.8.3 Why Vanilla JavaScript

The frontend was deliberately implemented without a component framework such as React or Vue. For an application of this scope — a set of related list, form, and dashboard views — a framework's build tooling and abstraction layer were judged to add more complexity than they would remove. Vanilla JavaScript keeps the compiled output identical to the source code, requires no build step, and allows the entire frontend to be understood by reading a small number of plain files, which was considered valuable both for maintainability and for the educational goals of the project.

3.9 Security Design

Several deliberate design decisions were made to protect user data:

- PIN credentials are never stored in plain text; they are hashed using bcrypt before being written to the database, so that even direct access to the database file does not reveal a user's PIN.
- Authentication state is maintained using server-side sessions rather than storing sensitive tokens in browser local storage, reducing exposure to client-side script injection.
- Every data-access route reads the authenticated user's identifier from the server-side session rather than trusting any value supplied by the client, which prevents one account from requesting another account's data by manipulating a request.
- File uploads (receipt photos) are handled through Multer with restrictions on file type and are stored outside of the publicly served application code.
- All destructive operations, such as deleting a customer or clearing all data, require explicit confirmation on the frontend before the corresponding API call is made.

Chapter 4: Technology Stack

This chapter describes every major technology used to build the system, the reasoning behind each choice, and the alternative options that were considered before settling on the final stack. The guiding principle throughout was to prefer a small number of mature, well-understood tools over a larger number of specialized libraries, in order to keep the codebase easy to read, debug, and extend.

4.1 Frontend Technologies

4.1.1 HTML5

What it is: HTML5 is the standard markup language used to structure the single-page application shell, including the sidebar navigation, page containers, forms, and modal dialogs.

Why it was selected: HTML5 was the natural choice as the foundation of any browser-based interface. Its semantic elements and native form controls (date pickers, number inputs, file inputs) reduced the amount of custom UI code required for common tasks such as selecting a due date or uploading a receipt photo.

Alternatives considered: There was no realistic alternative to HTML as the base markup layer; the only real decision was whether to author it directly, as was done here, or generate it through a templating engine or component framework.

4.1.2 CSS3

What it is: CSS3 is used for all visual styling, including the layout grid system, color theme, responsive breakpoints for mobile devices, and print-specific styles used by the PDF export feature.

Why it was selected: A hand-written CSS3 stylesheet, organized around a small set of reusable design tokens (colors, spacing, and radius variables), gave full control over the visual identity of the application without introducing a CSS framework's default styling that would need to be overridden.

Alternatives considered: CSS frameworks such as Bootstrap or Tailwind CSS were considered but were judged unnecessary given the relatively small and consistent set of UI patterns (cards, tables, forms, modals) used throughout the application.

4.1.3 JavaScript (Vanilla, ES6+)

What it is: Modern JavaScript, using ES6+ features such as async/await, template literals, and arrow functions, implements all client-side logic: page navigation, form handling, API communication, and dynamic rendering of lists, tables, and dashboard widgets.

Why it was selected: Vanilla JavaScript avoids the build tooling, compilation step, and additional dependency surface of a frontend framework, which keeps the frontend directly readable and easy to debug in the browser without source maps.

Alternatives considered: React and Vue.js were considered, since both would offer more structured state management for a growing interface. They were set aside in favor of vanilla JavaScript because the application's page count and interaction complexity did not justify the additional build tooling and learning overhead for this project's scope.

4.1.4 Chart.js

What it is: Chart.js is a lightweight charting library used to render the bar, line, and doughnut charts that appear in the dashboard and the monthly and yearly report views, including revenue-versus-expense comparisons, daily cash flow trends, and payment-method breakdowns.

Why it was selected: Chart.js integrates directly with a plain HTML canvas element and requires no build step, which matched the rest of the frontend stack. Its declarative configuration object made it straightforward to keep chart styling consistent with the application's color theme.

Alternatives considered: D3.js was considered for its greater flexibility, but was judged to require significantly more code to achieve the same standard chart types already provided out of the box by Chart.js.

4.2 Backend Technologies

4.2.1 Node.js

What it is: Node.js is the JavaScript runtime that executes the backend server code outside of a browser environment.

Why it was selected: Using Node.js allowed the same programming language, JavaScript, to be used across both the frontend and backend, reducing context-switching during development

and allowing certain formatting and validation logic to be shared conceptually between client and server.

Alternatives considered: Backend runtimes based on other languages, such as Python with Flask or Django, or PHP, were considered. Node.js was preferred specifically to keep a single-language codebase.

4.2.2 Express.js

What it is: Express.js is the minimal web application framework used to define HTTP routes, apply middleware (authentication, JSON parsing, file uploads), and return JSON responses to the frontend.

Why it was selected: Express.js is described in detail in Section 3.8.2; in summary, it was chosen for its minimal footprint and its close alignment with the resource-oriented route structure of the application.

Alternatives considered: Fastify and Koa were considered as lighter or more modern alternatives, but Express.js's larger ecosystem of compatible middleware (including Multer and express-session, both used in this project) made it the more practical choice.

4.3 Database

4.3.1 SQLite (via better-sqlite3)

What it is: SQLite is a self-contained, file-based relational database engine. The project accesses it through the better-sqlite3 Node.js driver, which provides a synchronous query interface well suited to the request-response nature of the Express.js route handlers.

Why it was selected: SQLite requires no separate database server process, which simplifies both local development and eventual deployment, while still providing full SQL query support, foreign key constraints, and transactional writes. This made it possible to replace the project's original JSON-file storage with a properly normalized schema (see Section 3.8.1) without introducing the operational overhead of running a database server.

Alternatives considered: PostgreSQL and MySQL were considered for their support of concurrent multi-process access, but were judged unnecessary for the target deployment scale of a small business application, where a single embedded database file is sufficient.

4.4 Supporting Libraries

4.4.1 bcryptjs

What it is: bcryptjs implements the bcrypt password hashing algorithm in pure JavaScript, used to hash and verify the four-digit PIN associated with each user account.

Why it was selected: bcrypt-based hashing is a widely trusted, purpose-built algorithm for password storage, incorporating a per-hash salt and a configurable work factor that resists brute-force attacks even against short PINs.

Alternatives considered: Simple hashing algorithms such as SHA-256 were considered but rejected, since general-purpose hash functions are not designed to resist brute-force attacks against short, low-entropy secrets such as a four-digit PIN.

4.4.2 express-session

What it is: express-session provides server-side session management, issuing a session cookie to the browser after a successful login and using it to identify the authenticated user on every subsequent request.

Why it was selected: Session-based authentication was chosen over storing an authentication token directly in browser storage, reducing the surface area for client-side token theft and keeping the authentication state fully under server control.

Alternatives considered: Token-based authentication schemes were considered but were judged to add unnecessary complexity for a system where the frontend and backend are served from the same origin.

4.4.3 Multer

What it is: Multer is Express.js middleware for handling multipart/form-data, used specifically to accept image uploads for transaction receipts and expense documentation.

Why it was selected: Multer integrates directly with Express.js route handlers and provided sufficient configuration (file size limits, storage destination) for the project's modest file-upload requirements without needing a dedicated object storage service.

Alternatives considered: Cloud storage services such as Amazon S3 were considered for storing uploaded images, but were set aside in favor of local disk storage to avoid introducing a paid external dependency for a feature used relatively infrequently.

4.4.4 UUID

What it is: The uuid package generates globally unique identifiers used as primary keys for customers, transactions, products, batches, suppliers, and other core records.

Why it was selected: Using generated UUIDs instead of auto-incrementing integer keys made it straightforward to create related records (for example, a product and its first inventory batch) in a single logical operation without needing to first read back an auto-generated integer ID.

Alternatives considered: Auto-incrementing integer primary keys, SQLite's default option, were considered and would have worked equally well; UUIDs were preferred for consistency with the JSON-based data model used earlier in the project's development.

4.5 Development and Version Control Tools

4.5.1 Git and GitHub

What it is: Git is a distributed version control system used to track changes to the codebase throughout development; GitHub is the remote hosting platform used to store the repository.

Why it was selected: Git provided a complete history of the project's evolution, including the migration from JSON-file storage to SQLite and the incremental addition of each feature module, which made it possible to review, and if necessary revert, individual changes in isolation.

Alternatives considered: Other version control systems such as Mercurial were not seriously considered, given Git's status as the de facto standard for software projects of this kind.

4.6 Technology Stack Summary

Table 4.1 summarizes the complete technology stack used to build the system.

Layer	Technology	Purpose
Structure	HTML5	Page and component markup
Styling	CSS3	Visual design, layout, and responsiveness
Client Logic	JavaScript (ES6+)	Interactivity, API calls, and rendering
Data Visualization	Chart.js	Dashboard and report charts
Runtime	Node.js	Backend execution environment
Web Framework	Express.js	Routing and middleware
Database	SQLite (better-sqlite3)	Persistent relational data storage
Security	bcryptjs	PIN hashing
Authentication	express-session	Session-based login state
File Handling	Multer	Receipt and document image uploads
Identifiers	uuid	Unique primary key generation
Version Control	Git / GitHub	Source code management

Table 4.1: Technology Stack Summary

Chapter 5: Implementation

This chapter describes how each major feature of the system was implemented, covering its purpose, the user-facing workflow, the underlying backend, frontend, and database mechanics, and any notable implementation challenge encountered along with the solution adopted. Features are presented in roughly the order a shop owner would encounter them while using the application.

5.1 Customer Management

Purpose: Allows the shop owner to maintain a directory of customers who purchase on credit, forming the foundation that every sale and repayment is recorded against.

Workflow: The shop owner opens the Customers page, searches or scrolls to find an existing customer, or adds a new one by entering a name, phone number, address, and an optional credit limit. Each customer card displays their current balance and a computed trust score at a glance, and can be opened for a detailed transaction history.

Technical Implementation: On the backend, the customers table stores each record scoped to the authenticated user's ID. Balances are not stored directly; they are computed on read by summing the customer's debit (credit sale) and credit (repayment) transactions, which guarantees the displayed balance can never drift out of sync with the transaction history. The trust score is calculated from the customer's repayment ratio, the proportion of dues repaid on or before their due date, and a penalty for average repayment delay. On the frontend, a searchable dropdown component allows quickly locating a customer by name or phone number when recording a transaction elsewhere in the application.

Challenge and Solution: Initially, customer balances were stored as a cached field that was updated whenever a transaction was recorded. This proved error-prone, since a bug in any single code path that modified a balance would silently corrupt that customer's data. The solution was to remove the cached balance entirely and always compute it on demand from the full transaction history, trading a small amount of computation for a strong correctness guarantee.

5.2 Supplier Management

Purpose: Mirrors the customer ledger concept for the shop's own suppliers, allowing the shop owner to track how much they owe wholesalers for stock purchased on credit.

Workflow: The shop owner records a supplier once, then logs a debit whenever stock is purchased on credit from that supplier and a credit whenever a payment is made toward the outstanding balance. The supplier detail page presents the same balance-and-history pattern used for customers, reinforcing a consistent mental model throughout the application.

Technical Implementation: The suppliers and supplier_transactions tables follow the same structural pattern as customers and transactions. Purchasing inventory on credit from a linked supplier automatically creates a corresponding debit entry in the supplier ledger, removing the need for the shop owner to record the same purchase twice.

Challenge and Solution: A challenge arose in keeping the supplier ledger and the inventory batch records consistent when a purchase involved a supplier. The solution was to perform the inventory batch insertion and the supplier transaction insertion within a single database transaction, so that either both records are created successfully or neither is, preventing partial, inconsistent updates.

5.3 Inventory Management

Purpose: Tracks the products a shop sells, along with stock quantity, cost price, selling price, and expiry date, while presenting a single, uncluttered row per product regardless of how many separate purchases contributed to its current stock.

Workflow: When new stock is purchased, the shop owner either creates a new product or adds a new purchase batch to an existing product, entering the quantity, cost price, and optional expiry date for that specific batch. The inventory list displays one row per product showing total stock, average cost price, and nearest expiry date, with an expandable section revealing the individual batches that make up that total.

Technical Implementation: Two tables support this feature: products, which stores the shared attributes of a product (name, selling price, branch), and inventory_batches, which stores each individual purchase with its own remaining quantity, cost price, and expiry date. When a sale is recorded, stock is deducted automatically using a first-in-first-out strategy based on expiry

date, so that older or sooner-to-expire stock is sold first; the shop owner may also manually select a specific batch when recording a sale if required.

Challenge and Solution: The main challenge was preventing the inventory list from becoming cluttered when the same product was purchased multiple times at different prices, which was the exact behavior of the original flat inventory table. The solution was the two-table product-and-batch design described above, which separates what is displayed (an aggregated product row) from what is stored (individual batches), resolved through an aggregation query executed whenever the inventory page is loaded.

5.4 Sales Recording

Purpose: Provides the core, most frequently used action in the application: recording that a customer purchased goods, either for immediate cash payment or on credit.

Workflow: The shop owner opens the Add Transaction form, selects a customer, optionally selects a product and quantity (which automatically calculates the sale amount from the product's selling price and pulls the current stock level for validation), chooses whether the sale is cash or credit, and submits the form.

Technical Implementation: A cash sale is recorded as a transaction of type cash_sale and simultaneously increases the balance of the relevant cash or mobile-service account. A credit sale is recorded as a transaction of type debit and increases the customer's outstanding balance without any immediate movement of cash. Both transaction types, when linked to a product, trigger the FIFO stock deduction logic described in Section 5.3.

Challenge and Solution: A challenge was ensuring that a sale could not be completed if it would oversell available stock, particularly when multiple batches were involved. The solution was to validate the requested quantity against the product's aggregated available stock before beginning the deduction process, and to perform the deduction and the transaction insertion inside a single database transaction so that a failure partway through leaves no partial stock deduction behind.

5.5 Credit Management

Purpose: Represents the portion of a sale that a customer has not yet paid for, and is the central concept that distinguishes this system from a simple point-of-sale application.

Workflow: Every credit sale increases a customer's outstanding balance and may optionally be given a due date. The customer list and detail views highlight customers with a positive balance, and the Overdue page specifically surfaces customers whose due date has passed without full repayment.

Technical Implementation: Outstanding balance is never stored as a static number; it is computed as the sum of all debit transactions minus the sum of all credit transactions for that customer, as described in Section 5.1. Due dates are stored on the individual debit transaction, allowing the overdue calculation to identify which specific unpaid sales, rather than which customers in general, have passed their due date.

Challenge and Solution: A challenge was accurately identifying overdue amounts when a customer had several separate credit sales with different due dates and had made partial repayments. The solution, described further in Section 5.6, was to apply repayments to the oldest outstanding debit first, which keeps the notion of "overdue" meaningful at the level of an individual sale rather than only at the level of a customer's total balance.

5.6 Repayment System

Purpose: Allows the shop owner to record a customer paying down some or all of their outstanding balance, updating the ledger accordingly.

Workflow: From a customer's detail page, or from the Add Transaction form in repayment mode, the shop owner enters the amount received and, optionally, the payment method used (cash, bKash, Nagad, Rocket, or bank). The system immediately reduces the customer's outstanding balance and reflects the payment in the relevant account balance.

Technical Implementation: When a repayment is recorded, the backend iterates through the customer's outstanding debit transactions in chronological order, applying the repayment amount to the oldest unpaid debit first until the repayment is exhausted or all debits are settled. Each affected debit transaction stores its own repaidAmount and, once fully settled, a repaidAt timestamp, which allows the trust score calculation described in Section 5.1 to measure repayment speed at the level of an individual sale.

Challenge and Solution: A challenge was correctly handling a repayment that only partially covers one debit while fully covering another. The solution was to track repaidAmount as a running total on each debit transaction rather than treating a debit as a single all-or-nothing

unit, allowing repayments to be distributed across multiple debits of varying ages within a single operation.

5.7 Expense Management

Purpose: Allows the shop owner to record money spent on running the business, separate from money owed by customers, which is necessary to calculate genuine profit rather than only revenue.

Workflow: The shop owner records an expense with a title, category (for example, Rent, Salary, Transportation, or Utility), amount, date, and payment method, optionally attaching a photo of the receipt.

Technical Implementation: Expenses are stored in a dedicated expenses table with a category field used throughout the reporting module to break down spending by type. When a payment method is specified, the expense also deducts from the corresponding account balance, keeping the account ledger consistent with actual cash movement.

Challenge and Solution: A challenge was ensuring expense categories remained consistent enough to produce a meaningful category breakdown in reports, while still allowing the shop owner flexibility in naming individual expenses. The solution was to constrain the category field to a fixed set of common categories selected from a dropdown, while leaving the expense title itself as free text.

5.8 Dashboard

Purpose: Gives the shop owner a single-glance overview of the business without requiring navigation into individual pages, intended to be the first screen seen after logging in.

Workflow: On login, the dashboard loads and displays total receivable and total collected amounts, an overdue summary, account balances across cash and mobile-service accounts, a list of recent transactions, a short-term income-versus-expense trend chart, and a ranked list of customers with the highest outstanding balances.

Technical Implementation: The dashboard is served by a single aggregating API endpoint that queries multiple tables and returns a combined summary object, rather than requiring the frontend to make many separate requests. The top-due-customers list and trend chart are

rendered using the same underlying calculation logic used elsewhere in the reporting module, ensuring the dashboard's numbers always agree with the detailed reports.

Challenge and Solution: A challenge was keeping the dashboard responsive despite aggregating data from several tables on every load. The solution was to scope every dashboard query to the authenticated user and, where applicable, the currently selected branch, keeping the volume of data processed per request proportional to a single shop's data rather than the entire system.

5.9 Reports (Monthly and Yearly)

Purpose: Provides structured, period-based financial summaries that would be impractical to produce by hand from a paper ledger, giving the shop owner insight into trends over time.

Workflow: For the Monthly Report, the shop owner selects a year and month and is shown total revenue, total expense, net profit, a daily cash-flow chart, product and customer analytics, and a breakdown of expenses by category and payments by method. For the Yearly Report, the shop owner selects a year and is shown a month-by-month grouped chart of revenue, expense, and profit, alongside year-level key performance indicators and a small set of automatically generated business insights, such as the best-performing month or the most active customer.

Technical Implementation: Both reports are generated by dedicated backend endpoints that query the transactions and expenses tables for the relevant date range, scoped to the authenticated user and, if selected, a specific branch. All chart rendering is handled on the frontend using Chart.js, with the backend responsible only for producing the underlying numeric summaries.

Challenge and Solution: A challenge was ensuring that terminology used in the reports matched what a Bangladeshi shop owner would actually expect. An early version of the monthly report labeled total cash collected as "Revenue" (রাজস্ব), a formal accounting term; after review, this was corrected to "Collections" (আদায়), a term that matches how shop owners naturally describe money received, without changing the underlying calculation.

5.10 CSV Export

Purpose: Allows a shop owner to download report data in spreadsheet form for offline analysis, printing outside the application, or sharing with an accountant.

Workflow: From either the Monthly Report or Yearly Report screen, the shop owner selects a CSV export option, and a file is generated and downloaded directly by the browser without any additional server round-trip beyond the data already loaded for the report.

Technical Implementation: CSV generation is implemented entirely on the client using a small set of JavaScript helper functions that assemble report data into properly escaped comma-separated rows, taking care to quote any field containing a comma, quotation mark, or line break, and to prefix the file with a UTF-8 byte-order mark so that Bengali text displays correctly when the file is opened in spreadsheet applications such as Microsoft Excel. The resulting file is delivered to the user through the browser's Blob and object URL APIs, triggering a standard file download.

Challenge and Solution: A challenge was ensuring Bengali characters in customer names and shop names did not appear as garbled text when the exported file was opened in Excel. The solution was to prefix the generated CSV content with a UTF-8 byte-order mark before creating the download blob, which allows Excel to correctly detect the file's text encoding.

5.11 PDF Export

Purpose: Allows a shop owner to produce a print-ready, formatted document version of a monthly or yearly report, suitable for printing or archiving as a PDF file.

Workflow: The shop owner selects a PDF export option from a report screen, which opens a new browser tab containing a specially formatted version of the report, styled for printing. The browser's native print dialog then opens automatically, and the shop owner chooses "Save as PDF" as the destination to produce a PDF file, or sends it directly to a physical printer.

Technical Implementation: Rather than relying on a server-side PDF generation library, the system builds a complete, self-contained HTML document on the client, including inline print-specific CSS rules that control page size, margins, and page-break behavior, and injects the relevant report data (summary figures, tables, and chart images captured from the on-screen canvas elements) directly into that document before opening it in a new window and invoking the browser's print function. This approach reuses the same HTML and CSS skills already applied throughout the rest of the frontend, rather than introducing a separate PDF-generation dependency on the backend.

Challenge and Solution: A challenge was making sure tables did not visually overflow or split awkwardly across page boundaries when printed. The solution was a dedicated print stylesheet that disables page breaks inside table rows and key summary sections, and repeats table headers at the top of each new printed page.

5.12 Search

Purpose: Allows the shop owner to quickly locate a specific customer or product without scrolling through a potentially long list.

Workflow: A search box above the customer list filters visible customers as the shop owner types, matching against both name and phone number. Similar searchable selection is used for products and customers inside the Add Transaction form.

Technical Implementation: Search is implemented entirely on the client side against data already loaded for the current page, avoiding a network request on every keystroke. For the transaction form specifically, a reusable searchable-dropdown component was built that filters a supplied list of items by one or more fields and supports keyboard navigation (arrow keys and Enter) in addition to mouse selection.

Challenge and Solution: A challenge arose when this searchable dropdown component was first introduced: selecting a result did not reliably store the selected item's identifier in the hidden field the save logic depended on, causing transactions to fail silently. The solution was to make the component explicitly write the selected item's identifier and related attributes onto the hidden input's dataset whenever a selection is made, and to validate that a valid identifier is present before allowing a transaction to be saved.

5.13 Filtering

Purpose: Allows the shop owner to narrow down transaction lists and reports by criteria such as branch, date range, transaction type, and payment method.

Workflow: On the Transactions page, the shop owner can filter by date, transaction type (cash sale, credit sale, or repayment), and payment method. Across the application, a branch selector in the sidebar filters nearly every page — dashboard, customers, inventory, and reports — to show only data belonging to the selected branch, or all branches combined.

Technical Implementation: Filtering is applied primarily through query parameters passed to the relevant backend endpoint, which appends the corresponding SQL WHERE conditions before executing the query, ensuring that filtering happens efficiently at the database level rather than by discarding rows on the client after they have already been transferred.

Challenge and Solution: A challenge was keeping filter selections consistent across page navigation, so that switching from the Dashboard to the Customers page while a specific branch was selected did not silently reset back to "All Branches." The solution was to persist the currently selected branch in the browser's local storage and read it back whenever a page is loaded.

5.14 Settings

Purpose: Provides a central location for the shop owner to manage account-level configuration that does not belong to any single business record.

Workflow: From the Settings page, the shop owner can edit their shop name and owner name, add or remove branches, change their login PIN, and trigger a full data backup or restore.

Technical Implementation: Shop name, owner name, and the branch list are stored in the app_settings table, one row per user account. Branch deletion is guarded against removing the last remaining branch and automatically reassigns any customers, inventory, or expenses associated with a deleted branch to the shop's default branch, rather than leaving them without a valid branch reference. PIN changes require the current PIN to be verified before a new one is accepted, and the new PIN is hashed using the same bcrypt-based process used at registration.

Challenge and Solution: A challenge encountered during development was that the shop name and owner name fields initially appeared editable but did not persist changes, because the corresponding save handler was not correctly wired to the input fields. The solution involved auditing the form for stale event bindings and disabled attributes left over from an earlier version of the page, and rebuilding the save flow to explicitly read current input values, submit them to the settings endpoint, and update the active session with the confirmed response.

5.15 Authentication

Purpose: Ensures that each shop owner's data is private and that only the correct account can access it, while keeping the login process fast enough for daily repeated use.

Workflow: A new shop owner registers with their name, shop name, a phone number or email address, and a four-digit PIN. To log in, they enter the same identifier and PIN. Once authenticated, the shop owner remains logged in for the duration of their browser session.

Technical Implementation: PINs are hashed with bcrypt before storage, and never stored or transmitted in plain text after registration. Successful login establishes a server-side session using express-session, and a middleware function checks for a valid session on every protected route, attaching the authenticated user's ID to the request so that all subsequent database queries are automatically scoped to that user.

Challenge and Solution: A challenge was supporting login by either phone number or email without duplicating the login logic. The solution was to accept a single identifier field on the login form and have the backend attempt to match it against both the phone and email columns of the users table before falling back to a failed-login response.

5.16 Image Upload

Purpose: Allows the shop owner to attach a photo, such as a receipt or an expense document, to a transaction or expense record for future reference.

Workflow: While recording a repayment or an expense, the shop owner may optionally select an image file from their device. After the underlying transaction or expense record is saved, the image is uploaded and linked to that record.

Technical Implementation: Image uploads are handled by Multer middleware on a dedicated upload route, which validates file type and stores the file on the server's local disk under an uploads directory, saving only the resulting file path in the database rather than the binary image data itself. The frontend displays a thumbnail wherever an attached image exists and allows opening the full image in a modal.

Challenge and Solution: A challenge was avoiding orphaned image files if a transaction failed to save after an image had already been selected. The solution was to always create the underlying transaction or expense record first, and only perform the image upload as a second, separate request once a valid record identifier exists to associate the image with.

5.17 Database Operations

Purpose: Encompasses the shared data-access patterns used throughout the backend to read, write, and maintain consistency across the SQLite database.

Workflow: Every feature described in this chapter ultimately reads from or writes to the SQLite database through a shared connection module. Multi-step operations, such as recording a sale that both deducts inventory and creates a transaction, or a repayment that touches several debit records, are wrapped in explicit database transactions.

Technical Implementation: The better-sqlite3 driver's synchronous transaction() function is used to group related statements so that they either all succeed or all fail together, preventing partially applied changes. Indexes were added on frequently filtered columns, including userId, customerId, and date, to keep list and report queries responsive as the volume of transaction data grows.

Challenge and Solution: A challenge was migrating existing data safely when the database schema changed, most notably when the system moved from a single-owner data model to a multi-user model requiring a userId column on every table. The solution was a migration routine, run automatically at server startup, that adds any missing columns, assigns existing data to a default owner account, and creates any dependent default records, such as the standard set of cash and mobile-service accounts, without requiring manual intervention or risking data loss.

5.18 Error Handling

Purpose: Ensures that failures, whether caused by invalid input, a missing record, or an unexpected server error, are communicated clearly to the shop owner rather than causing the application to silently fail or crash.

Workflow: When an operation fails, the shop owner sees a short, specific message describing what went wrong, presented as a temporary toast notification, rather than a generic error or a blank screen.

Technical Implementation: Backend route handlers wrap their database and validation logic in try-catch blocks and return a consistent JSON error shape with an appropriate HTTP status code. The frontend's shared API helper function inspects every response for this error shape

and throws a JavaScript error carrying the server's message, which is caught by the calling page script and displayed through a shared toast notification function.

Challenge and Solution: A challenge was avoiding generic, unhelpful error messages such as "something went wrong" for every failure. The solution was to ensure that validation logic on the backend returns a specific, human-readable message for each distinct failure case, for example, distinguishing insufficient stock from a missing customer selection, so that the shop owner immediately understands what needs to be corrected.

5.19 Validation

Purpose: Prevents invalid or inconsistent data from being recorded in the first place, complementing the error handling described in Section 5.18.

Workflow: Forms throughout the application check required fields, numeric ranges, and logical constraints before allowing a submission to proceed, for example, preventing a sale from being recorded without a positive amount, or preventing a quantity greater than the available stock.

Technical Implementation: Validation is applied at two layers: lightweight checks on the frontend give immediate feedback without a network round-trip, while the backend independently re-validates every request before writing to the database, since frontend validation alone cannot be trusted to prevent invalid data if the API is called directly. Examples include verifying that a PIN is exactly four digits, that a branch being deleted is not the shop's last remaining branch, and that a repayment amount is a positive number.

Challenge and Solution: A challenge was keeping frontend and backend validation rules consistent so that a form which appeared to accept an input did not then fail unexpectedly on the server. The solution was to treat the backend validation rules as the single source of truth and mirror the same constraints on the frontend purely as a usability convenience, rather than maintaining two independently evolving sets of rules.

Chapter 6: Testing

6.1 Manual Testing Approach

Given the size of the project and the absence of a dedicated automated testing framework, testing was carried out manually and iteratively throughout development rather than as a single phase at the end. Each feature was tested immediately after implementation using realistic data, and previously working features were re-checked whenever a related change was made elsewhere in the codebase, particularly after the migration from JSON-file storage to SQLite and again after the introduction of multi-user support.

Testing focused on three levels: verifying that individual actions produced the correct immediate result (for example, that recording a credit sale increased the correct customer's balance by the correct amount), verifying that calculated values remained consistent across different views of the same data (for example, that a customer's balance shown on the dashboard matched the balance shown on their detail page), and verifying that the system behaved safely under invalid input, such as attempting to sell more stock than was available.

6.2 Test Environment

Testing was performed using the Node.js development server running locally, with the SQLite database file reset to a known state before each significant round of testing using the application's built-in demo data seeding endpoint. The application was tested in a modern desktop web browser, with periodic checks of the responsive layout using the browser's mobile device emulation to confirm usability on smaller screens, since shop owners are expected to frequently access the system from a mobile phone.

6.3 Test Cases

Table 6.1 lists a representative sample of the manual test cases executed against the system.

TC ID	Test Scenario	Expected Result	Status
TC-01	Register a new account with a valid phone number and PIN	Account is created and the user is redirected to log in	Pass
TC-02	Register a new account with a phone number already in use	Registration is rejected with a clear duplicate-account message	Pass

TC ID	Test Scenario	Expected Result	Status
TC-03	Log in with the correct phone number and PIN	User is authenticated and redirected to the dashboard	Pass
TC-04	Log in with an incorrect PIN	Login is rejected with an invalid-credentials message	Pass
TC-05	Add a new customer with a name and phone number	Customer appears in the customer list with a zero balance	Pass
TC-06	Record a credit sale for a customer	Customer balance increases by the sale amount	Pass
TC-07	Record a cash sale linked to a product	Product stock decreases by the sold quantity; no change to customer balance	Pass
TC-08	Attempt to sell more units than are currently in stock	Sale is rejected with an insufficient-stock message	Pass
TC-09	Record a partial repayment against a customer with multiple debits	Repayment is applied to the oldest unpaid debit first	Pass
TC-10	Add a new purchase batch to an existing product	Product's total stock and average cost price update; a new batch appears in the expanded view	Pass
TC-11	Delete a branch that still has customers assigned to it	Deletion is blocked, or affected customers are reassigned to the default branch	Pass
TC-12	Attempt to delete the last remaining branch	Deletion is rejected with an explanatory message	Pass
TC-13	Generate a monthly report for a month with no transactions	Report loads with all totals shown as zero, without errors	Pass
TC-14	Generate a yearly report and open the CSV export	A correctly formatted CSV file downloads with accurate totals	Pass
TC-15	Open the PDF export for a monthly report	A print-formatted preview opens and the browser print dialog appears	Pass
TC-16	Search for a customer by partial name	Matching customers are filtered in real time	Pass

TC ID	Test Scenario	Expected Result	Status
TC-17	Change the account PIN using an incorrect current PIN	PIN change is rejected	Pass
TC-18	Log in as a second, separate account and view the dashboard	Only the second account's own data is visible	Pass
TC-19	Upload a receipt photo while recording an expense	Expense is saved with a viewable thumbnail attached	Pass
TC-20	Restore data from a previously exported JSON backup	Existing data is replaced with the backup's customers and transactions	Pass

Table 6.1: Manual Test Cases

6.4 Validation Testing

In addition to functional test cases, a set of validation-focused checks were performed to confirm that the system correctly rejected invalid input rather than silently accepting it. Table 6.2 summarizes the validation rules checked and the outcome of testing each one.

Validation Rule	Test Input	Expected Result	Actual Result
PIN must be exactly four digits	PIN of three digits	Rejected with a length error	Matched expected result
Transaction amount must be positive	Amount of zero or a negative number	Rejected before the transaction is saved	Matched expected result
Customer must be selected before saving a transaction	Transaction form submitted with no customer chosen	Rejected with a customer-required message	Matched expected result
Sale quantity cannot exceed available stock	Quantity greater than current stock	Rejected with an insufficient-stock message	Matched expected result
Repayment amount must be positive	Repayment of zero	Rejected before saving	Matched expected result
Duplicate phone number is rejected at signup	Signup using an already-registered phone number	Rejected with a duplicate-account message	Matched expected result

Validation Rule	Test Input	Expected Result	Actual Result
Shop name cannot be saved as empty	Settings form submitted with an empty shop name field	Rejected, previous shop name retained	Matched expected result

Table 6.2: Validation Test Summary

6.5 Summary of Testing Results

All test cases executed during manual testing produced the expected result at the time of final review, and no outstanding defects were known to remain unresolved in the core sale, repayment, inventory, and reporting workflows. Several defects were identified and corrected during earlier stages of testing, most notably an issue where the searchable customer and product selection components did not reliably store the selected item's identifier (resolved as described in Section 5.12), and an issue where shop name and owner name changes in Settings did not persist (resolved as described in Section 5.14). These issues were caught specifically because manual testing was performed continuously during development rather than only at the end of the project, reinforcing the value of the iterative testing approach adopted for this project.

Chapter 7: Results

7.1 Overview of the Completed Application

The completed হালখাতা ডিজিটাল (Halkhata Digital) application implements the full set of features described in Chapter 5: customer and supplier management, cash and credit sales recording, a repayment system with automatic FIFO allocation, batch-aware inventory tracking, expense recording, a summary dashboard, monthly and yearly financial reports with charts, CSV and PDF export, multi-branch support, and PIN-based authentication with per-account data isolation.

The interface is presented entirely in Bengali, using Bengali numeral formatting for currency and quantities throughout, which was a deliberate design decision to match the language and conventions the target users are most comfortable with. The application is fully responsive and was verified to remain usable on a small mobile screen, reflecting the expectation that most shop owners will primarily access the system from a smartphone rather than a desktop computer.

The following section presents representative screens from the completed application. Each figure is marked as a placeholder to be replaced with an actual screenshot of the running system before final submission.

7.2 Application Screenshots



Figure 7.1: Login Screen



Figure 7.2: Registration Screen

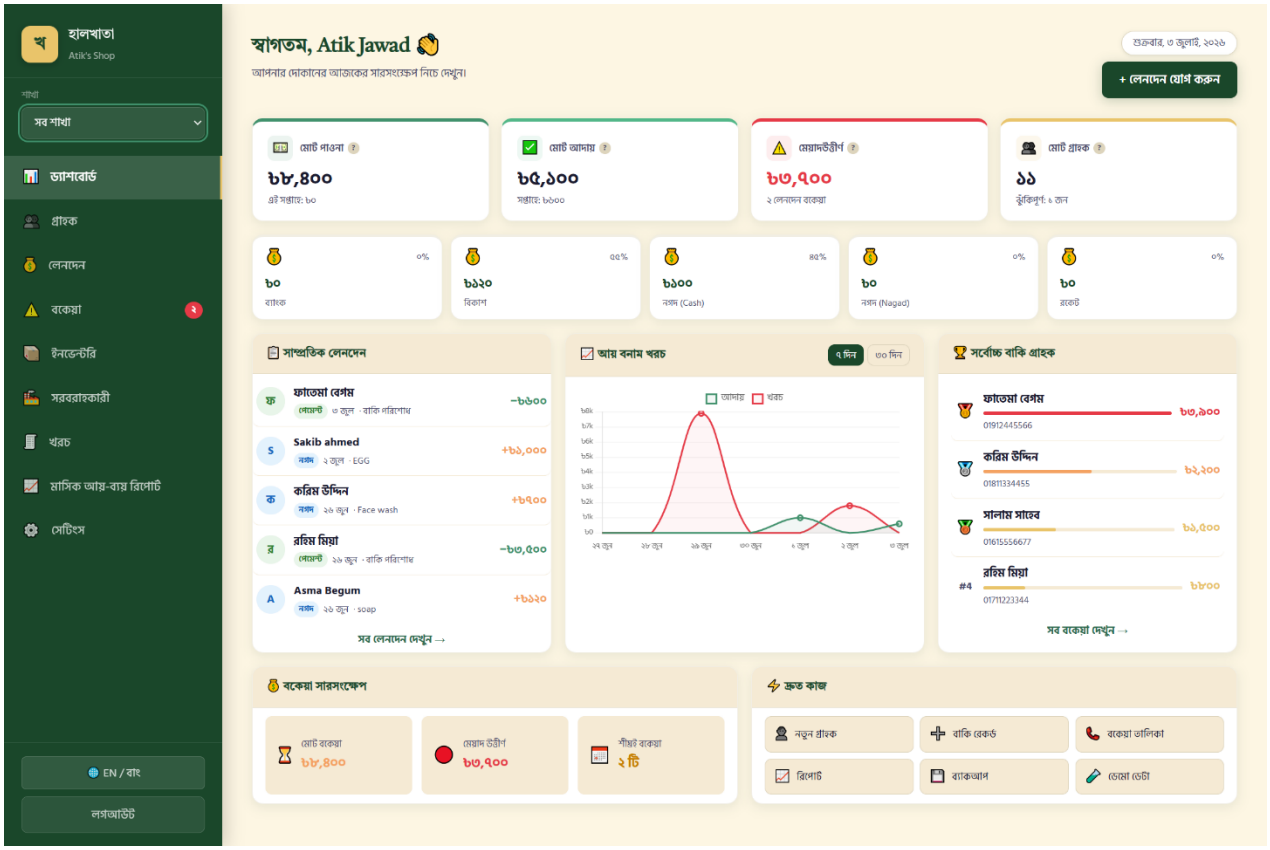


Figure 7.3: Dashboard Overview

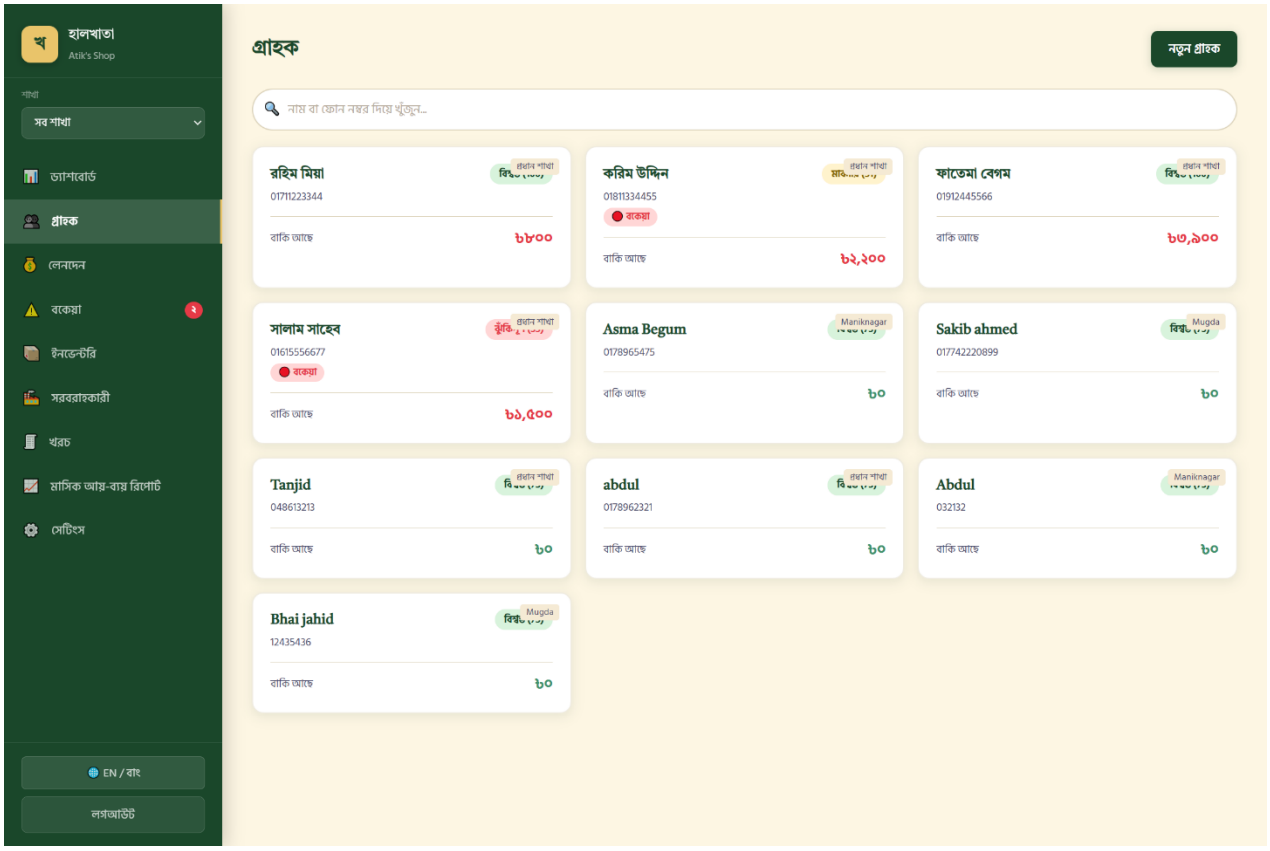


Figure 7.4: Customer Management Page

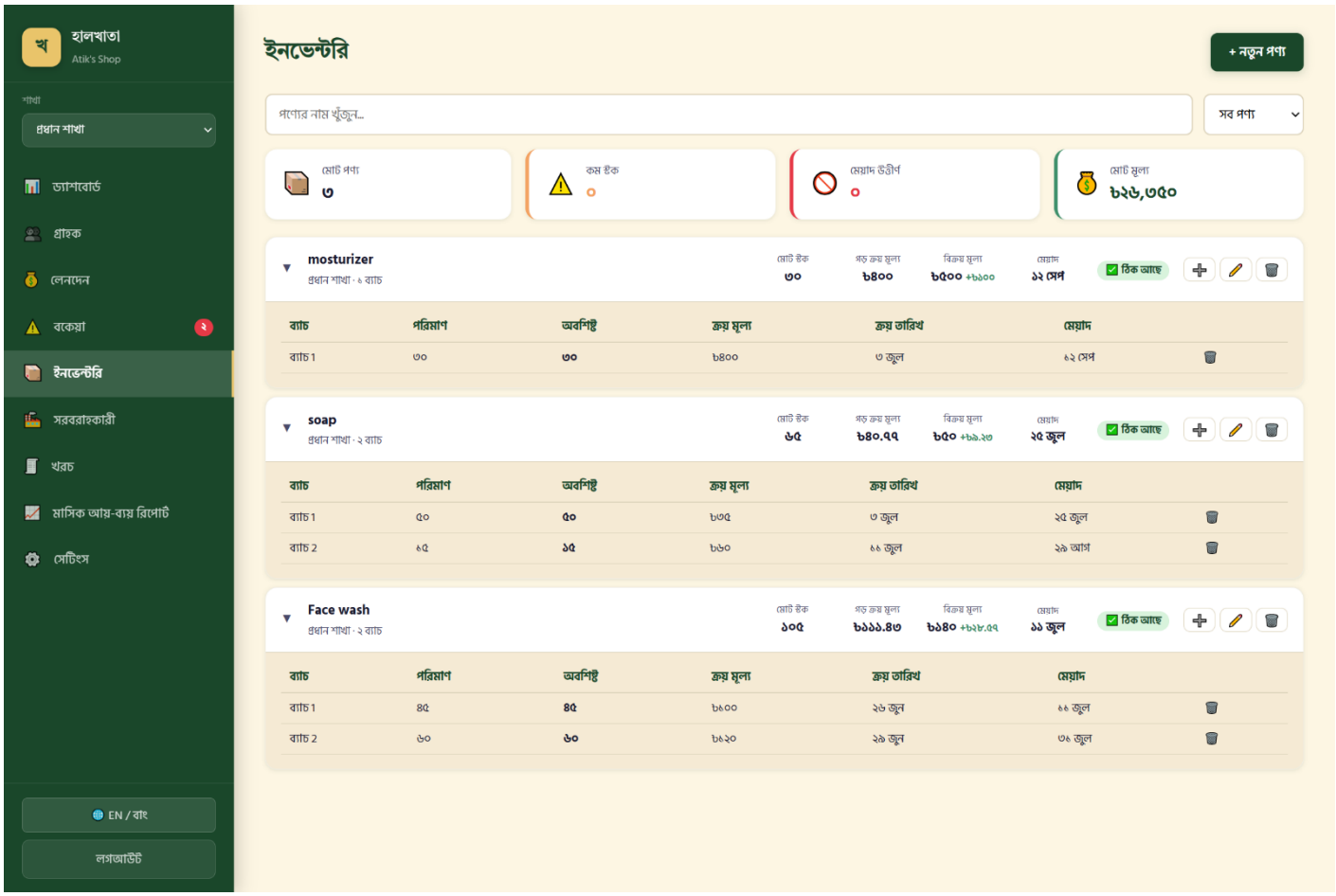


Figure 7.5: Inventory Management Page

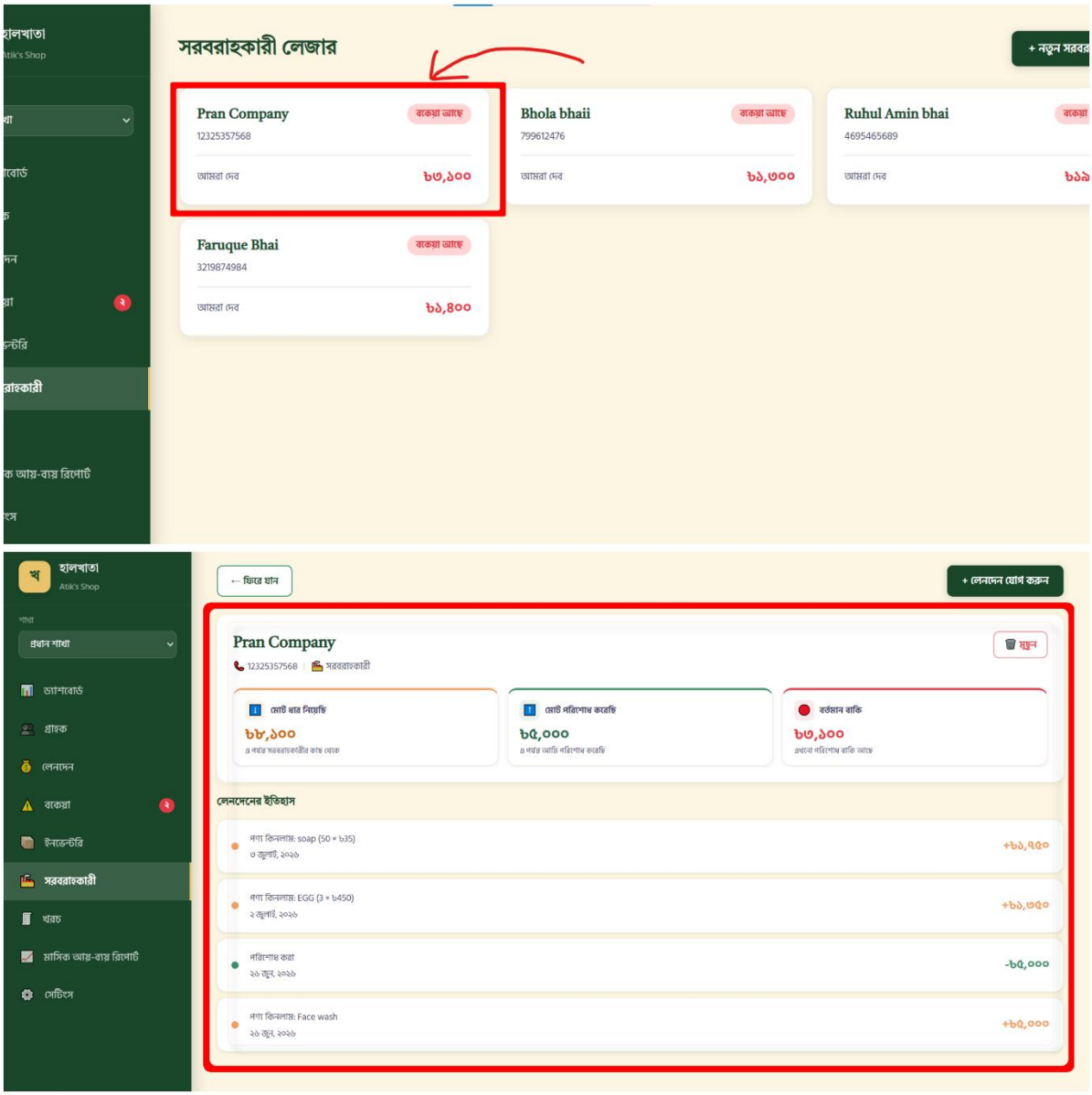


Figure 7.6: Supplier Ledger Page

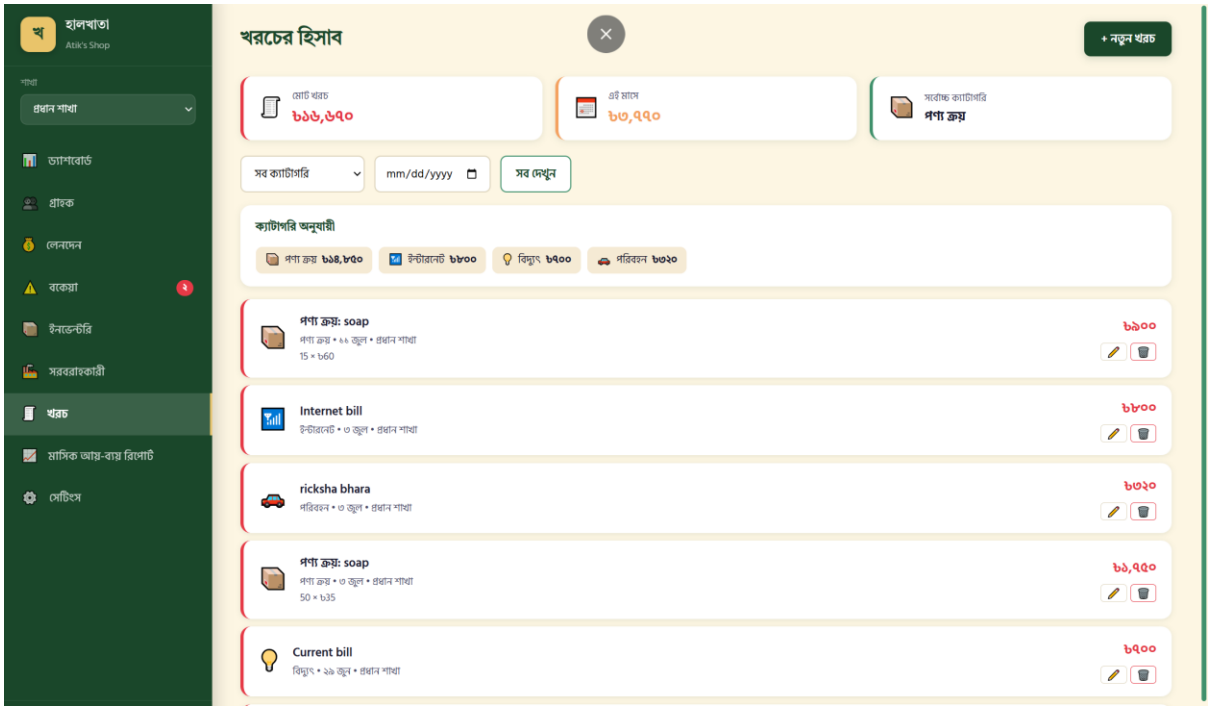


Figure 7.7: Expense Overview Page

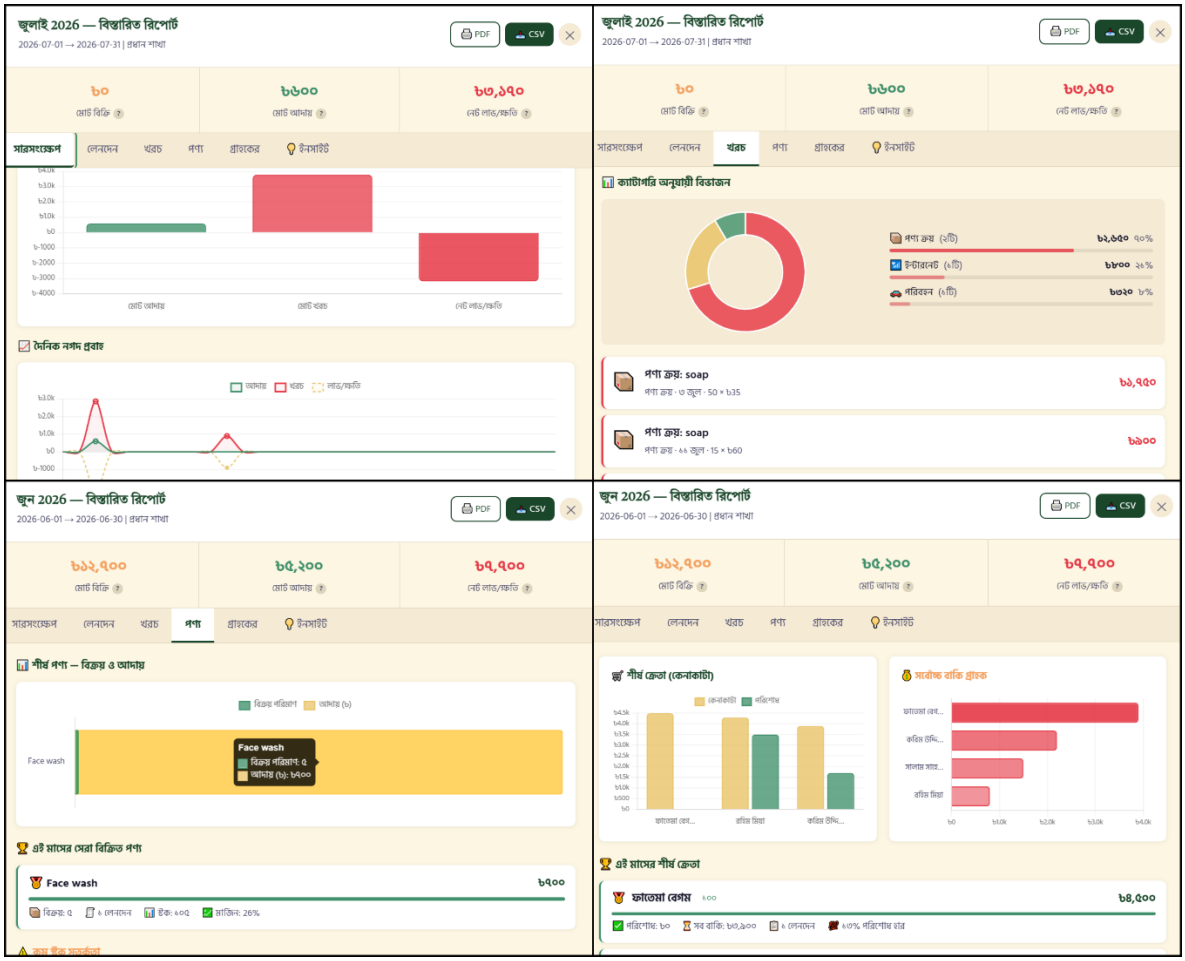


Figure 7.8: Monthly Report View

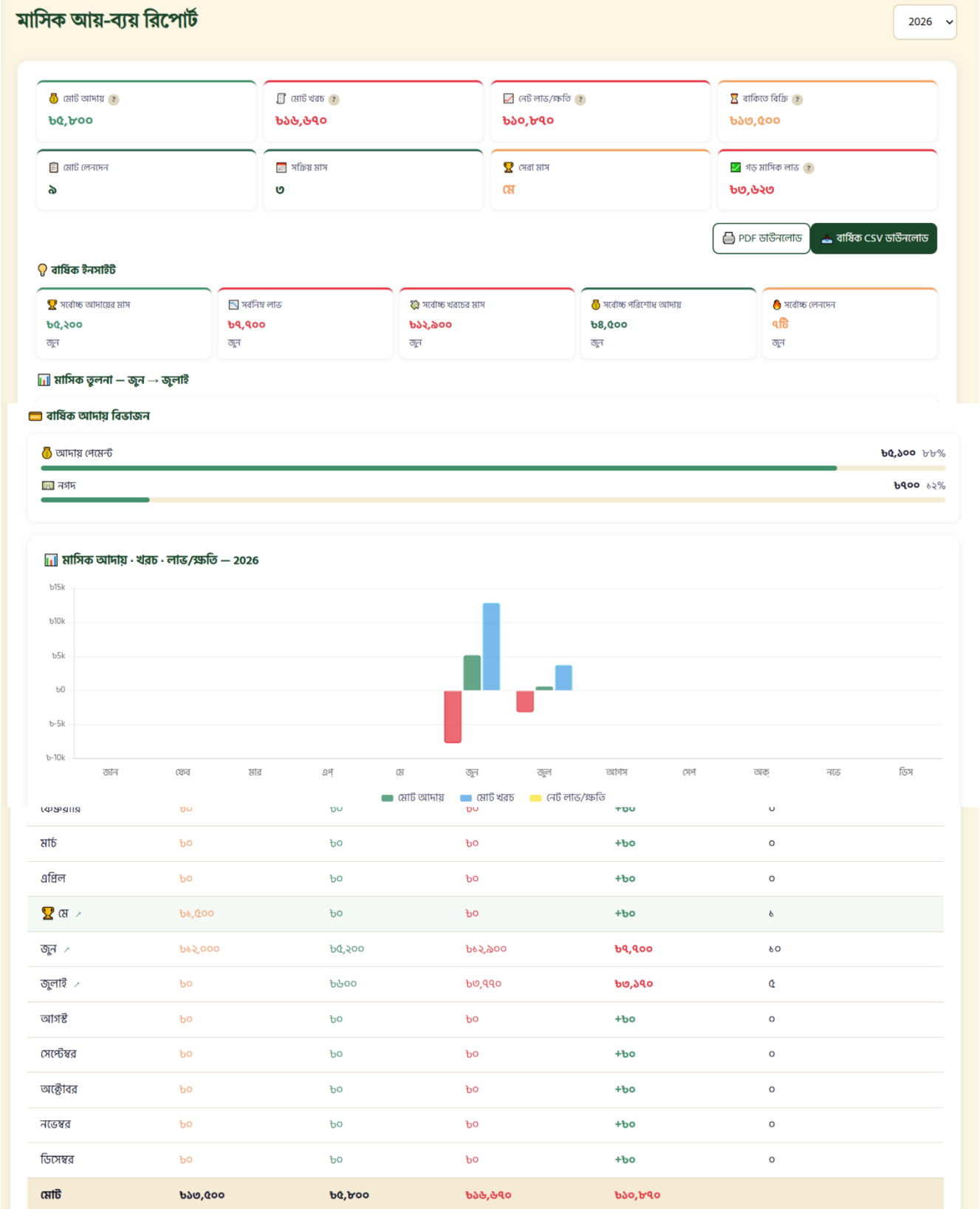


Figure 7.9: Yearly Report View

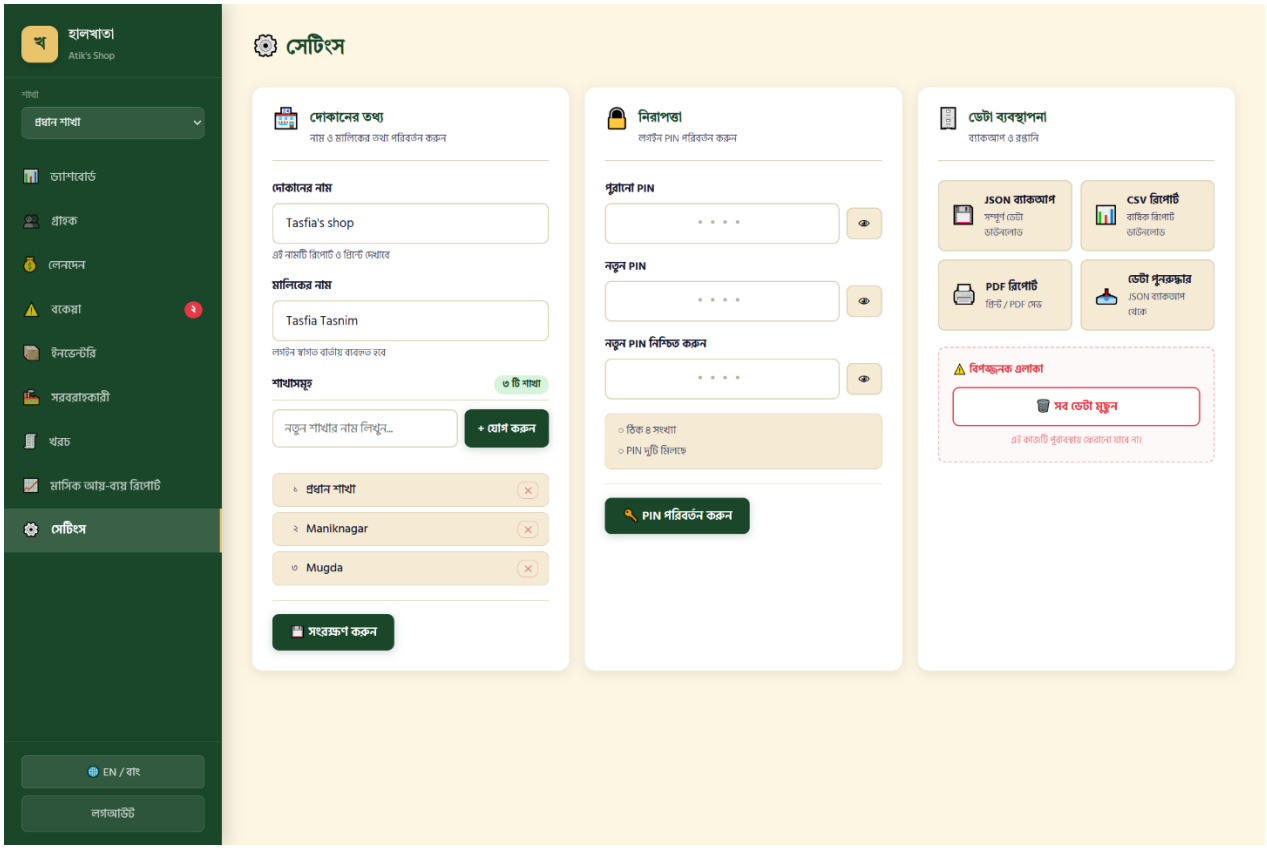
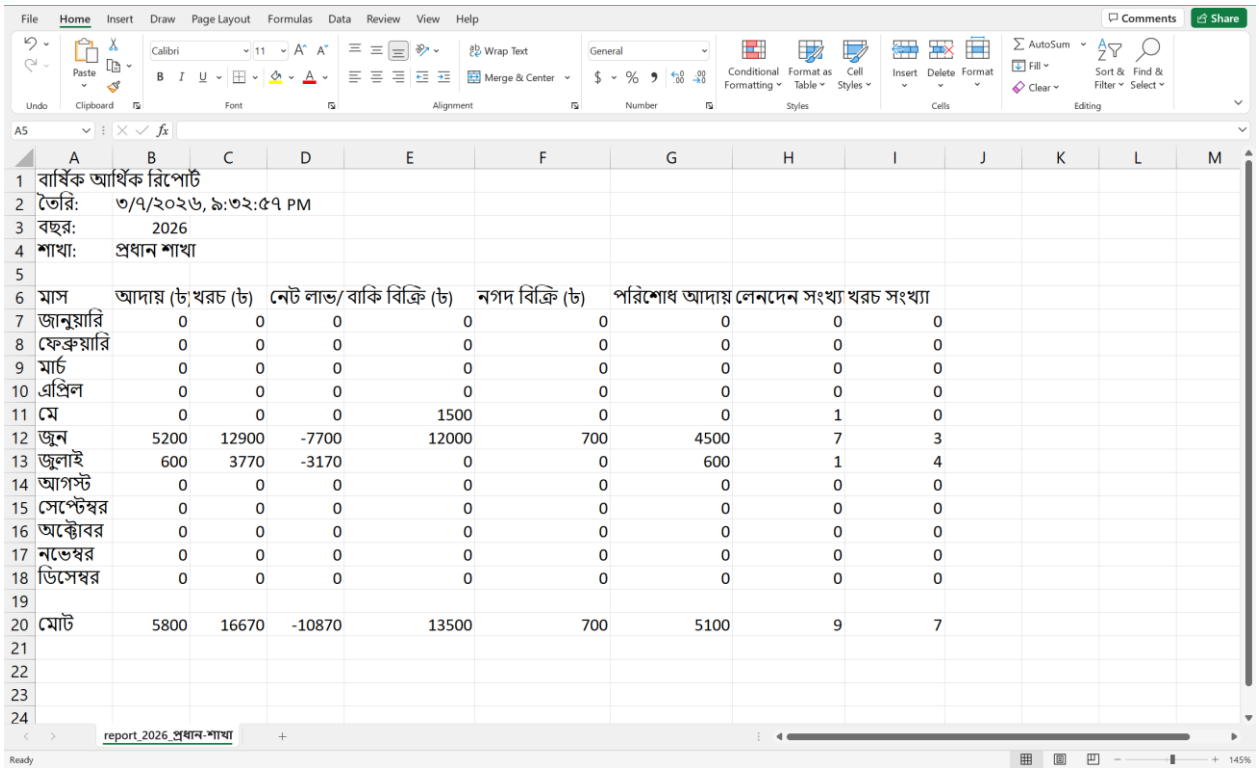


Figure 7.10: Settings Page



	A	B	C	D	E	F	G	H	I	J	K	L	M
1	বার্ষিক আর্থিক রিপোর্ট												
2	তৈরি:	৩/৭/২০২৬, ৯:৩২:৫৭ PM											
3	বছর:	2026											
4	শাখা:	প্রধান শাখা											
5													
6	মাস	আদায় (৳)	খরচ (৳)	নেট লাভ/	বাকি বিক্রি (৳)	নগদ বিক্রি (৳)	পরিশোধ আদায়	লেনদেন সংখ্যা	খরচ সংখ্যা				
7	জানুয়ারি	0	0	0	0	0	0	0	0				
8	ফেব্রুয়ারি	0	0	0	0	0	0	0	0				
9	মার্চ	0	0	0	0	0	0	0	0				
10	এপ্রিল	0	0	0	0	0	0	0	0				
11	মে	0	0	0	1500	0	0	1	0				
12	জুন	5200	12900	-7700	12000	700	4500	7	3				
13	জুলাই	600	3770	-3170	0	0	600	1	4				
14	আগস্ট	0	0	0	0	0	0	0	0				
15	সেপ্টেম্বর	0	0	0	0	0	0	0	0				
16	অক্টোবর	0	0	0	0	0	0	0	0				
17	নভেম্বর	0	0	0	0	0	0	0	0				
18	ডিসেম্বর	0	0	0	0	0	0	0	0				
19													
20	মোট	5800	16670	-10870	13500	700	5100	9	7				
21													
22													
23													
24													

Figure 7.11: CSV Export Result

Chapter 8: Conclusion

8.1 Summary

This project set out to replace the traditional paper হালখাতা with a digital equivalent that a small Bangladeshi shop owner could adopt without abandoning the workflow they already understand. The resulting system, হালখাতা ডিজিটাল (Halkhata Digital), was built using HTML5, CSS3, and vanilla JavaScript on the frontend, Node.js and Express.js on the backend, and SQLite for persistent storage, and implements the complete lifecycle of small-business credit management: customer records, cash and credit sales, repayments, batch-aware inventory, a supplier ledger, expense tracking, and structured monthly and yearly financial reporting with export to CSV and PDF.

8.2 Achievements

- Successfully migrated the application's storage layer from flat JSON files to a normalized, relational SQLite schema without losing existing data, improving both data integrity and query performance.
- Implemented an automatic FIFO-based repayment allocation system that applies a customer's payment to their oldest outstanding debit first, closely matching how a shop owner would reason about repayment by hand.
- Designed and implemented a batch-aware inventory model that keeps the visible product list uncluttered while preserving accurate cost and expiry information for every individual purchase.
- Delivered a working multi-user system in which each registered shop owner's data, including customers, transactions, inventory, and settings, is fully isolated from every other account.
- Built monthly and yearly financial reports with interactive charts, product and customer analytics, and CSV and PDF export, entirely from first-party code without relying on a third-party reporting service.
- Produced a fully Bengali-language interface with locally relevant payment method tracking (cash, bKash, Nagad, Rocket, and bank), directly reflecting the needs of the target user base.

8.3 Limitations

As with any student project developed within a limited timeframe, the current system has a number of known limitations. It requires an active internet connection to the hosting server for every operation and does not currently support offline data entry. There is no native mobile application; the system is accessed exclusively through a mobile web browser. Each account represents a single shop owner, and there is no mechanism for multiple staff members to share an account with different permission levels. The system has not been tested under a large-scale, concurrent multi-user production load, and its current SQLite-based storage, while well suited to a single-shop deployment, would need to be re-evaluated for a very large number of simultaneous businesses.

8.4 Lessons Learned

Developing this project reinforced the importance of designing a data model around the true source of a value rather than caching a derived value that must then be kept manually in sync — a lesson learned directly from the early, error-prone approach of storing customer balances as a cached field, discussed in Section 5.1. It also demonstrated the practical benefit of migrating from an ad hoc storage format to a properly normalized relational database as an application's complexity grows, rather than attempting to postpone that migration indefinitely. Finally, the project highlighted the value of terminology review specific to the target audience: a system can be functionally correct while still confusing its users if it uses formal or foreign terminology, such as an early version's use of "Revenue" instead of the more locally natural "Collections" described in Section 5.9, in place of language the user actually expects.

Chapter 9: Future Work

While the current system fulfils its core objective of digitizing the halkhata workflow, several enhancements were identified during development as valuable directions for future iterations of the project. These were deliberately excluded from the current scope to keep the initial implementation focused and achievable within the project timeline, but are realistic extensions of the existing architecture.

- **Offline Support:** Enabling core actions such as recording a sale to be performed without an active internet connection, with automatic synchronization once connectivity is restored, would improve reliability in areas with unstable mobile data coverage.
- **Android Application:** A native Android application would provide a faster, more integrated experience than the current mobile browser interface, and could support features such as home-screen shortcuts and push notifications.
- **iOS Application:** A companion iOS application would extend the system's reach to shop owners using Apple devices.
- **Barcode Scanner Support:** Allowing products to be identified by scanning a printed barcode using a phone camera would speed up both inventory entry and point-of-sale product selection.
- **QR Code Support:** Generating a QR code for a customer's account could allow faster lookup during a sale, and could support digital verification of a customer's identity.
- **Cloud Synchronization:** Extending the current backup-and-restore feature into automatic, continuous cloud synchronization would protect shop owners against data loss without requiring a manual export.
- **SMS Notifications:** Sending an automatic SMS reminder to customers with an approaching or overdue due date could improve repayment rates without requiring the shop owner to contact each customer manually.
- **Dark Mode:** Adding a dark color theme would improve usability in low-light conditions and align with a common expectation of modern mobile applications.

- **Advanced Analytics:** Deeper analytical features, such as customer segmentation, seasonal demand trends, or product profitability ranking over longer time horizons, would build on the reporting foundation already in place.
- **AI-Based Prediction:** Predictive features such as forecasting future cash flow or flagging a customer at elevated risk of default before they become overdue could be built on top of the existing trust score and transaction history data.
- **Multiple Employee Accounts:** Introducing role-based permissions would allow a shop owner to grant limited access to trusted staff, for example permitting them to record sales without granting access to financial reports or settings.
- **Multi-Language Support:** Although the interface is currently fully localized in Bengali, adding an English-language toggle would broaden the system's usability for a wider range of users and business types.

Each of these directions builds naturally on the existing architecture described in Chapter 3 rather than requiring a fundamental redesign, which supports the feasibility of extending the system incrementally in future work.